

Phase 1 - React Fundamentals and Application Shell

Phase objective

The objective of Phase 1 was to replace the generated Vite demo with the first real version of the Quiz Learning Platform interface while learning the React fundamentals required to build and explain the application professionally.

At the end of this phase, the application includes:

- A reusable application header
- A home screen
- A hero section with a quiz creation action
- A quiz configuration panel
- A controlled study-content form
- Accessible difficulty selection
- A local score dashboard using mock data
- Derived score calculations
- Responsive styling
- Component and utility tests

The application does not generate a real quiz yet. The current quiz submission logs a configuration object that will later be sent to the backend.

Product decisions made during this phase

Keep the product general

The application is especially useful for Data Analysis and Portuguese, but the interface does not restrict the learner to those subjects.

Instead of forcing the user to select a hard-coded subject, the form asks one natural question:

What did you learn?

The learner can describe any topic, for example:

- SQL joins, aggregation, and HAVING
- Portuguese greetings and verb conjugations
- Statistics, history, programming, or another subject

The backend will later infer the subject and construct an appropriate prompt for the language model.

Keep the first quiz configuration simple

The form currently collects only:

- Study content
- Difficulty

The first version will use a fixed question count. This reduces friction and keeps the first user experience focused.

Explanations are required

Every generated quiz question will eventually include:

- The correct answer
- A concise explanation of why it is correct

This is part of the learning value of the product and will be represented as a required field in the quiz domain model.

Scores appear on the home screen

The home screen displays progress information to make learning feel competitive and motivating.

The current score data is temporary mock data. In a later phase, it will be replaced with scores stored in `localStorage`.

Final component structure

```
apps/web/src/
|-- components/
|   |-- AppHeader.tsx
|   |-- PrimaryButton.tsx
|-- features/
|   |-- home/
|   |   |-- HeroSection.tsx
|   |   |-- HomeScreen.tsx
|   |-- quiz-generation/
|   |   |-- QuizConfigurationForm.tsx
|   |   |-- QuizConfigurationPanel.tsx
|   |   |-- quizConfiguration.types.ts
|   |-- scores/
|   |   |-- mockScores.ts
|   |   |-- quizScore.types.ts
|   |   |-- ScoreBoard.tsx
|   |   |-- score.utils.ts
|   |   |-- score.utils.test.ts
|-- test/
|   |-- setup.ts
|-- App.css
```

```
|-- App.test.tsx
|-- App.tsx
|-- index.css
`-- main.tsx
```

This is a small feature-oriented structure. It separates reusable UI, home-specific UI, quiz-generation code, and score-related code without adding unnecessary abstraction layers.

React component model

A React component is a function that returns UI markup.

```
function AppHeader() {
  return (
    <header className="app-header">
      <a href="/">Quiz Learning Platform</a>
    </header>
  );
}
```

Component names begin with an uppercase letter. Lowercase JSX names are interpreted as native HTML elements.

Component composition

The application is composed as a tree:

```
App
|-- AppHeader
`-- HomeScreen
    |-- HeroSection
    |   `-- PrimaryButton
    `-- ScoreBoard
```

A larger screen is built by combining smaller components with clear responsibilities.

Props and one-way data flow

Props are values passed from a parent component to a child component.

```
<HomeScreen
  scores={mockScores}
  onCreateQuiz={handleOpenQuizConfiguration}
/>
```

The data flows downward:

```
App
  ↓ props
HomeScreen
  ↓ props
ScoreBoard
```

The child component does not modify the parent directly. It receives data and callbacks through props.

Why `HomeScreen` receives scores through props

`HomeScreen` originally imported `mockScores` directly. That coupled the component to a temporary data source.

The improved version receives scores through props:

```
interface HomeScreenProps {
  scores: QuizScore[];
  onCreateQuiz: () => void;
}
```

This means `HomeScreen` does not care whether the scores come from:

- Mock data
- `localStorage`
- A backend API
- A database

This improves reuse, testing, and maintainability.

Callback props

A parent can provide behavior to a child through a callback prop.

```
interface HeroSectionProps {
  onCreateQuiz: () => void;
}
```

The child calls the function when the user interacts with the UI:

```
<PrimaryButton onClick={onCreateQuiz}>
  Create a quiz
</PrimaryButton>
```

The child knows that the user requested quiz creation, but it does not need to know how the application changes screens.

This preserves separation of responsibilities.

State with `useState`

State is data owned by a component that can change over time and cause React to render again.

```
const [isQuizConfigurationOpen, setIsQuizConfigurationOpen] =  
  useState(false);
```

The state value is initially `false`, so the home screen appears.

```
setIsQuizConfigurationOpen(true);
```

When the value changes to `true`, React renders the quiz configuration panel.

State update cycle

```
User interaction  
  ↓  
Event handler executes  
  ↓  
State setter is called  
  ↓  
React renders again  
  ↓  
The UI reflects the new state
```

Conditional rendering

React can select which interface to render based on state.

```
{isQuizConfigurationOpen ? (  
  <QuizConfigurationPanel  
    onClose={handleCloseQuizConfiguration}  
    onSubmit={handleQuizConfigurationSubmit}  
  />  
) : (  
  <HomeScreen  
    scores={mockScores}  
    onCreateQuiz={handleOpenQuizConfiguration}  
  />  
)}  
)}
```

The ternary operator follows this structure:

```
condition ? valueWhenTrue : valueWhenFalse
```

Only one of the two screens is rendered at a time.

Controlled form inputs

A controlled input receives its value from React state and reports user changes through an event handler.

```
const [studyContent, setStudyContent] = useState("");

<textarea
  value={studyContent}
  onChange={handleStudyContentChange}
/>

function handleStudyContentChange(
  event: ChangeEvent<HTMLTextAreaElement>,
): void {
  setStudyContent(event.target.value);
}
```

The interaction cycle is:

```
User types
  ↓
`onChange` executes
  ↓
React state changes
  ↓
The component renders again
  ↓
The textarea displays the updated state
```

Form submission

The form prevents the browser's default page reload:

```
function handleSubmit(
  event: FormEvent<HTMLFormElement>,
): void {
  event.preventDefault();

  onSubmit({
    studyContent: studyContent.trim(),
    difficulty,
  });
}
```

The form emits a small domain object:

```
export interface QuizConfiguration {
  studyContent: string;
  difficulty: QuizDifficulty;
}
```

TypeScript union types

Difficulty is modeled as a finite set of allowed values:

```
export type QuizDifficulty =
  | "beginner"
  | "intermediate"
  | "advanced";
```

TypeScript rejects unsupported values such as:

```
const difficulty: QuizDifficulty = "impossible";
```

This makes invalid states harder to represent.

Accessible difficulty cards

The difficulty choices look like selectable cards, but their semantics are radio inputs.

```
<input
  type="radio"
  name="difficulty"
  value={option.value}
  checked={difficulty === option.value}
  onChange={handleDifficultyChange}
/>
```

Radio inputs are correct because only one difficulty can be selected.

The inputs share the same `name`, so the browser treats them as one group.

The group uses `fieldset` and `legend`:

```
<fieldset>
  <legend>Choose a difficulty</legend>
  { /* radio options */ }
</fieldset>
```

This gives assistive technologies the context that all three options answer the same question.

Rendering lists with map

Difficulty options and score history are collections. React renders them with `map`.

```
{difficultyOptions.map((option) => (  
  <label key={option.value}>  
    {/* option content */}  
  </label>  
))}  
  
{scores.map((score) => (  
  <li key={score.id}>  
    {score.quizTitle}  
  </li>  
))}
```

The purpose of key

A `key` helps React identify each item between renders.

A good key is:

- Unique among siblings
- Stable over time
- Derived from the item's identity

The score ID and difficulty value are better keys than an array index.

Derived values versus state

A derived value can be calculated from existing data.

Examples in this phase include:

- Score percentage
- Average score
- Personal best
- Number of completed quizzes
- Whether the Generate Quiz button is disabled

```
const isSubmitDisabled = studyContent.trim().length === 0;  
const averageScore = calculateAverageScore(scores);
```

These values should not normally be stored with `useState`, because the source data already exists.

Why average score is not state

The average is calculated from the score array. If it were also stored in state, the application would have two sources of truth:

Scores
Stored average

Those values could become inconsistent.

A useful Swift analogy is a computed property rather than `UserDefaults`:

```
var averageScore: Int {  
    calculateAverage(scores)  
}
```

`localStorage`, like `UserDefaults`, is persistence. It will later store the underlying scores. The average should still be computed from those scores.

Pure score functions

Score calculations were extracted into pure functions.

```
export function calculateScorePercentage(  
    score: Pick<QuizScore, "correctAnswers" | "totalQuestions">,  
) : number {  
    if (score.totalQuestions <= 0) {  
        return 0;  
    }  
  
    return Math.round(  
        (score.correctAnswers / score.totalQuestions) * 100,  
    );  
}
```

A pure function:

- Does not modify its input
- Does not depend on React
- Does not access browser storage
- Returns the same result for the same input

This makes business rules easy to test independently.

Avoiding duplicated data

`QuizScore` stores answer counts but not percentage:

```
export interface QuizScore {
  id: string;
  quizTitle: string;
  difficulty: QuizDifficulty;
  correctAnswers: number;
  totalQuestions: number;
  completedAt: string;
}
```

Percentage is calculated from `correctAnswers` and `totalQuestions`.

Storing all three could create contradictory values:

```
{
  correctAnswers: 8,
  totalQuestions: 10,
  percentage: 95,
}
```

The current model keeps one source of truth.

Mock data and progressive implementation

The score dashboard currently receives `mockScores`.

Mock data allows the team to:

- Build the interface before persistence exists
- Develop without a backend
- Create predictable tests
- Validate the product experience early

Later, App will obtain real scores from a storage abstraction or custom hook.

The expected transition is:

```
scores={mockScores}
```

becoming something similar to:

```
const { scores } = useQuizScores();
```

```
<HomeScreen
  scores={scores}
  onCreateQuiz={handleOpenQuizConfiguration}
/>
```

`HomeScreen` and `ScoreBoard` should remain unchanged because they already depend on the score data, not its source.

Semantic HTML and accessibility

The application uses semantic elements such as:

- `header`
- `main`
- `section`
- `h1`, `h2`, and `h3`
- `button`
- `form`
- `fieldset`
- `legend`
- `ol` and `li`

Inputs have labels connected through `htmlFor` and `id`.

Sections use `aria-labelledby` when appropriate.

Tests query controls by accessible role and name, which encourages an interface that works with assistive technology.

Responsive layout

The interface uses CSS Grid, flexible widths, and media queries.

Examples:

```
.home-screen {
  display: grid;
  gap: 4rem;
  width: min(100%, 72rem);
  margin: 0 auto;
}

@media (max-width: 48rem) {
  .difficulty-selector__options {
    grid-template-columns: 1fr;
  }
}
```

The layout supports desktop, tablet, and narrow mobile widths without a separate mobile application.

Testing strategy

Component tests

`App.test.tsx` verifies behavior visible to the user:

- The home screen is displayed initially
- The score dashboard is present
- The configuration screen opens
- Quiz generation is disabled without study content
- Study content and difficulty are submitted correctly

React Testing Library queries elements through semantic roles and labels:

```
screen.getByRole("button", {
  name: /generate quiz/i,
});
```

Utility tests

`score.utils.test.ts` verifies pure score calculations:

- Percentage calculation
- Zero-question handling
- Average calculation
- Personal-best calculation

Separating UI tests and business-rule tests keeps each test focused.

Problems encountered and lessons learned

CSS class mismatch

The header component used:

```
className="app-header"
```

while the stylesheet initially defined:

```
.app__header
```

Because class names must match exactly, the browser displayed the default link style.

Lesson: JSX and CSS selectors form an explicit contract.

Clicking a hidden radio input in a test

The visual difficulty card hides the radio input and disables pointer events on the input.

The original test attempted to click the hidden input directly, and `userEvent` correctly rejected the unrealistic interaction.

The improved test clicks the visible label and then verifies that the radio became selected.

Lesson: tests should simulate how users interact with the interface.

Scoreboard assertion in the wrong screen

A scoreboard assertion was temporarily placed inside a test that had already opened the configuration screen.

Because the application conditionally renders only one screen, the score dashboard was not present.

Lesson: understand the component state and current screen before writing an assertion.

Duplicate interactions in a test

A copied test temporarily clicked Intermediate and Generate Quiz twice.

The duplicate lines were removed after reviewing the complete test flow.

Lesson: review copied code as a sequence of user actions, not only as syntax.

Architecture lessons

Decoupling does not require excessive abstraction

Passing scores through props is a small, justified decoupling decision.

It solves a real upcoming change: replacing mock data with browser persistence.

The phase did not introduce repositories, dependency-injection containers, Redux, or a design-system package because the current application does not need them.

State should live in the nearest responsible owner

- `App` owns which screen is visible
- `QuizConfigurationForm` owns temporary form values
- `ScoreBoard` derives summary values from its props

External data sources should be isolated

Mock scores are currently provided by `App`. Later, browser storage will be loaded near the application boundary or inside a dedicated hook.

The presentational components do not need to change.

Senior-level discussion points

Why not use React Router yet?

The application currently has two temporary views and no requirement for shareable URLs, browser history, or deep links.

Local state is sufficient for Phase 1. React Router will be introduced when the application has real pages and URL-based navigation.

Why not use a form library yet?

The current form has one text area and one radio group. Native controlled inputs are simple and teach core React behavior.

A form library should be introduced when validation and form complexity justify the dependency.

Why not use global state?

All current state belongs to a small and clear component subtree.

Global state would add indirection without solving a real problem.

Why calculate scores outside React?

The calculations are domain rules rather than visual concerns. Pure TypeScript functions are easier to reuse and test.

Interview questions and model answers

What is the difference between props and state?

Props are values provided by a parent component. State is data owned by a component that can change and trigger a new render.

What is a controlled input?

A controlled input receives its displayed value from React state and updates that state through an event handler.

Why should derived values not normally be stored in state?

Because they can be calculated from existing data. Storing them creates duplicated sources of truth and synchronization risks.

What happens after a state setter is called?

React schedules an update, renders the component again, compares the new output with the previous output, and updates the necessary DOM elements.

Why does a list need keys?

Keys give list items stable identities so React can correctly reconcile additions, removals, and updates.

Why use radio inputs for difficulty?

Only one difficulty can be selected. Radio inputs provide the correct browser and accessibility semantics for a mutually exclusive group.

Why pass callbacks through props?

A child can report a user intention without owning the parent application's navigation or state-transition logic.

Why receive scores through props?

It keeps the component independent of the score source, making it easier to test, reuse, and migrate from mock data to persistence.

What is the difference between local state and persistence?

Local state exists while the React component or page is active. Persistence such as `localStorage` survives page reloads and browser sessions until it is removed.

When would you introduce a custom hook?

When related stateful behavior is reused or when extracting it makes a component easier to understand, for example loading and saving quiz scores.

Explain-back answers**Why does `HomeScreen` receive scores through props?**

Because it should depend on score data, not on the temporary source that provides the data. This makes the component easier to test and reuse.

Why is average score not stored with `useState`?

Because it is a derived value calculated from the score array. The raw scores may later be persisted in `localStorage`, but the average should still be calculated from them.

What changes when mock scores are replaced with `localStorage`?

The application boundary that obtains the scores changes, most likely `App` or a custom `useQuizScores` hook. `HomeScreen` and `ScoreBoard` should remain unchanged.

Development commands

Run all commands from the repository root:

```
C:\CAI\quiz-learning-platform
```

Select Node.js

```
nvm use 24.18.0
```

Install dependencies

```
npm install
```

For an exact clean installation based on `package-lock.json`:

```
npm ci
```

Run the application

```
npm run dev:web
```

Vite normally serves the application at:

```
http://localhost:5173
```

Format files

```
npm run format
```

Check formatting

```
npm run format:check
```

Run ESLint

```
npm run lint:web
```

Run TypeScript validation

```
npm run typecheck:web
```

Run tests once

```
npm run test:web
```

Run tests in watch mode

```
npm run test:watch --workspace @quiz-learning-platform/web
```

Create a production build

```
npm run build:web
```

Run the complete quality gate

```
npm run quality:web
```

Common Git workflow

```
git status
git add .
git commit -m "feat: add quiz configuration and score dashboard"
git push
```

Practice exercises

1. Change the default difficulty to Intermediate without reading the previous instructions.
 2. Add a fifth mock score and predict the new average before running the app.
 3. Add a `findLatestScore` pure function and write its unit tests.
 4. Add a character counter below the study-content textarea.
 5. Prevent submission when the trimmed study content has fewer than 20 characters.
 6. Add an empty score state by passing an empty array to `ScoreBoard`.
 7. Explain the complete data flow from `App` to `ScoreBoard`.
 8. Explain why `averageScore` behaves like a Swift computed property.
 9. Rewrite the difficulty options manually and explain why the `map` version is preferable.
 10. Describe which components should change when score persistence is introduced.
-

Personal study method

To avoid passive copy-and-paste development, use this sequence for each feature:

1. Read the problem and predict which component owns the state.
2. Implement one small file or behavior at a time.
3. Predict the result before running the application.
4. Explain the data flow in your own words.
5. Make one independent modification.
6. Run the quality gate.
7. Write a brief learning-journal entry.

A useful 90-minute session structure is:

15 minutes - Review the previous implementation
40 minutes - Implement the next behavior
15 minutes - Explain component ownership and data flow
10 minutes - Make an independent change
10 minutes - Test, document, and commit

Phase result

At the end of Phase 1, the application has a tested and responsive React shell with:

- Reusable components
- Props and callback props
- Local state
- Conditional rendering
- Controlled inputs
- Accessible radio-card selection
- List rendering
- Derived score calculations
- Pure utility functions
- Component tests
- Utility tests
- A home score dashboard

The project is ready for the next phase, where it will introduce the local quiz domain and playable quiz engine with fake quiz data before connecting to any AI provider.